(https://vscode.dev/github/microsoft/vscode-docs/blob/main/docs/copilot/copilot-customization.md)

Customize chat responses in VS Code

Copilot can provide you with responses that match your coding practices and project requirements if you give it the right context. Custom instructions enable you to define and automatically apply the guidelines and rules for tasks like generating code, or performing code reviews. Prompt files let you craft complete chat prompts in Markdown files, which you can then reference in chat or share with others. In this article, you will learn how to use custom instructions and prompt files to customize your chat responses in Visual Studio Code.

Enable instructions and prompt files in VS Code

To enable instructions and prompt files in VS Code, enable the ⚙ `chat.promptFiles (vscode://settings/chat.promptFiles)` setting.

To centrally enable or disable this setting within your organization, check Centrally Manage VS Code Settings (/docs/setup/enterprise#_centrally-manage-vs-code-settings) in the enterprise documentation.

Instruction files

Custom instructions enable you to describe common guidelines or rules to get responses that match your specific coding practices and tech stack. Instead of manually including this context in every chat query, custom instructions automatically incorporate this information with every chat request.

VS Code supports several types of custom instructions, targeted at different scenarios: code generation, test generation, code review, commit message generation, and pull request title and description generation instructions.

You can define custom instructions in two ways:

1. **Instruction files**: specify code-generation instructions in Markdown files for your workspace or VS Code profile (/docs/configure/profiles).

2 **Settings**: specify instructions in VS Code user or workspace settings.

Use instruction files

Instruction files enable you to specify custom instructions in Markdown files. You can use these files to define your coding practices, preferred technologies, and project requirements.

There are two types of instruction files:

- `.github/copilot-instructions.md` : a single instruction file that contains all the instructions for your workspace. These instructions are automatically included in every chat request.
- `.instructions.md` files: one or more prompt files that contain custom instructions for specific tasks. You can attach individual prompt files to a chat request, or you can configure them to be automatically included for specific files or folders.

If you specify both types of instruction files, both are included in the chat request. No particular order or priority is applied to the instructions. Make sure to avoid conflicting instructions in the files.

> Note
> Instruction files are only used for code generation and are not used for code completions (/docs/copilot/ai-powered-suggestions).

Use a .github/copilot-instructions.md file

You can store custom instructions in your workspace or repository in a `.github/copilot-instructions.md` file and describe your coding practices, preferred technologies, and project requirements by using Markdown. These instructions only apply to the workspace where the file is located.

VS Code automatically includes the instructions from the `.github/copilot-instructions.md` file to every chat request and applies them for generating code.

To use a `.github/copilot-instructions.md` file:

1 Set the
⚙ `github.copilot.chat.codeGeneration.useInstructionFiles`
`(vscode://settings/github.copilot.chat.codeGeneration.useInstructionFiles)`
setting to `true` to instruct Copilot in VS Code to use the custom instructions file.

2 Create a `.github/copilot-instructions.md` file at the root of your workspace. If needed, create a `.github` directory first.

3 Add natural language instructions to the file. You can use the Markdown format.

Whitespace between instructions is ignored, so the instructions can be written as a single paragraph, each on a new line, or separated by blank lines for legibility.

> Note

> GitHub Copilot in Visual Studio and GitHub.com also detect the `.github/copilot-instructions.md` file. If you have a workspace that you use in both VS Code and Visual Studio, you can use the same file to define custom instructions for both editors.

Use .instructions.md files

You can also create one or more `.instructions.md` files to store custom instructions for specific tasks. For example, you can create instruction files for different programming languages, frameworks, or project types.

VS Code supports two types of scopes for instruction files:

- **Workspace instructions files**: are only available within the workspace and are stored in the `.github/instructions` folder of the workspace.
- **User instruction files**: are available across multiple workspaces and are stored in the current VS Code profile (/docs/configure/profiles).

An instructions file is a Markdown file with the `.instructions.md` file suffix. The instruction file consists of two sections:

- (Optional) Header with metadata (Front Matter syntax)

Property	Description
`applyTo`	Specify a glob pattern for files to which the instructions are automatically applied. To always include the custom instructions, use the `**` pattern.

  For example, the following instructions file is always applied:

  ```
  ---
  applyTo: "**"
  ---
  Add a comment at the end of the file: 'Contains AI-generated edits.'
  ```

  Copy

- Body with the instruction content
  Specify the custom instructions in natural language by using Markdown formatting. You can use headings, lists, and code blocks to structure the instructions.
  You can reference other instruction files by using Markdown links. Use relative paths to reference these files, and ensure that the paths are correct based on the location of the instruction file.

To create an instructions file:

1. Run the **Chat: New Instruction File** command from the Command Palette (⇧⌘P).
2. Choose the location where the instruction file should be created.

   User instruction files are stored in the current profile folder (/docs/configure/profiles). You can sync your user instruction files across multiple devices by using Settings Sync (/docs/configure/settings-sync). Make sure to configure the **Prompts and Instructions** setting in the **Settings Sync: Configure** command.

Workspace instruction files are, by default, stored in the `.github/instructions` folder of your workspace. Add more instruction folders for your workspace with the ⚙ `chat.instructionsFilesLocations (vscode://settings/chat.instructionsFilesLocations)` setting.

3 Enter a name for your instruction file.

4 Author the custom instructions by using Markdown formatting.

Within a workspace instruction file, reference additional workspace files as Markdown links (`[index](../index.ts)`), or as `#index.ts` references within the instruction file.

To use an instructions file for a chat prompt, you can either:

- In the Chat view, select **Add Context** > **Instructions** and select the instruction file from the Quick Pick.
- Run the **Chat: Attach Instructions** command from the Command Palette (⇧⌘P) and select the instruction file from the Quick Pick.
- Configure the `applyTo` property in the instruction file header to automatically apply the instructions to specific files or folders.

Specify custom instructions in settings

You can also configure custom instructions in your user or workspace settings. There are specific settings for different scenarios. The instructions are automatically applied for the specific task.

The following table lists the settings for each type of custom instruction.

Type of instruction	Setting name
Code generation	⚙ `github.copilot.chat.codeGeneration.instructions (vscode://settings/github.copilot.chat.codeGeneration.instructions)`
Test generation	⚙ `github.copilot.chat.testGeneration.instructions (vscode://settings/github.copilot.chat.testGeneration.instructions)`
Code review	⚙ `github.copilot.chat.reviewSelection.instructions (vscode://settings/github.copilot.chat.reviewSelection.instructions)`
Commit message generation	⚙ `github.copilot.chat.commitMessageGeneration.instructions (vscode://settings/github.copilot.chat.commitMessageGeneration.instructions)`
Pull request title and description generation	⚙ `github.copilot.chat.pullRequestDescriptionGeneration.instructions (vscode://settings/github.copilot.chat.pullRequestDescriptionGeneration.instructions)`

You can define the custom instructions as text in the settings value or reference an external file in your workspace.

The following code snippet shows how to define a set of instructions in the `settings.json` file. To define instruction directly in settings, configure the `text` property. To reference an external file, configure the `file` property.

```
  "github.copilot.chat.codeGeneration.instructions": [
    {
      "text": "Always add a comment: 'Generated by Copilot'."
    },
    {
      "text": "In TypeScript always use underscore for private field names."
    },
    {
      "file": "general.instructions.md" // import instructions from file `general.instructi
ons.md`
    },
    {
      "file": "db.instructions.md" // import instructions from file `db.instructions.md`
    }
  ],
```

Copy

Custom instructions example

The following example demonstrates custom instructions for code generation:

- Define general coding guidelines in a `.github/instructions/general-coding.instructions.md` file that apply to all code:

  ```
  ---
  applyTo: "**"
  ---
  # Project general coding standards

  ## Naming Conventions
  - Use PascalCase for component names, interfaces, and type aliases
  - Use camelCase for variables, functions, and methods
  - Prefix private class members with underscore (_)
  - Use ALL_CAPS for constants

  ## Error Handling
  - Use try/catch blocks for async operations
  - Implement proper error boundaries in React components
  - Always log errors with contextual information
  ```

  Copy

- Define TypeScript and React coding guidelines in a `.github/instructions/typescript-react.instructions.md` file that apply to TypeScript and React code, general coding guidelines are inherited:

```
---
applyTo: "**/*.ts,**/*.tsx"
---
# Project coding standards for TypeScript and React

Apply the [general coding guidelines](./general-coding.instructions.md) to all code.

## TypeScript Guidelines
- Use TypeScript for all new code
- Follow functional programming principles where possible
- Use interfaces for data structures and type definitions
- Prefer immutable data (const, readonly)
- Use optional chaining (?.) and nullish coalescing (??) operators

## React Guidelines
- Use functional components with hooks
- Follow the React hooks rules (no conditional hooks)
- Use React.FC type for components with children
- Keep components small and focused
- Use CSS modules for component styling
```

Copy

Tips for defining custom instructions

- Keep your instructions short and self-contained. Each instruction should be a single, simple statement. If you need to provide multiple pieces of information, use multiple instructions.
- Don't refer to external resources in the instructions, such as specific coding standards.
- Split instructions into multiple files. This approach is useful for organizing instructions by topic or type of task.
- Make it easy to share custom instructions with your team or across projects by storing your instructions in instruction files. You can also version control the files to track changes over time.
- Use the `applyTo` property in the instruction file header to automatically apply the instructions to specific files or folders.
- Reference custom instructions in your prompt files to keep your prompts clean and focused, and to avoid duplicating instructions for different tasks.

Prompt files (experimental)

Prompt files allow you to craft complete prompts in Markdown files, which you can then reference in chat. Unlike custom instructions that supplement your existing prompts, prompt files are standalone prompts that you can store within your workspace and share with others. With prompt files, you can create reusable templates for common tasks, store domain expertise in your codebase, and standardize AI interactions across your team.

VS Code supports two types of scopes for prompt files:

- **Workspace prompt files**: are only available within the workspace and are stored in the `.github/prompts` folder of the workspace.

- **User prompt files**: are available across multiple workspaces and are stored in the current VS Code profile (/docs/configure/profiles).

Common use cases include:

- **Code generation**: create reusable prompts for components, tests, or migrations (for example, React forms, or API mocks).
- **Domain expertise**: share specialized knowledge through prompts, such as security practices, or compliance checks.
- **Team collaboration**: document patterns and guidelines with references to specs and documentation.
- **Onboarding**: create step-by-step guides for complex processes or project-specific patterns.

Prompt file structure

A prompt file is a Markdown file with the `.prompt.md` file suffix.

The prompt file consists of two sections:

- (Optional) Header with metadata (Front Matter syntax)

Property	Description
`mode`	The chat mode to use when running the prompt: `ask`, `edit`, or `agent` (default).
`tools`	The list of tools that can be used in agent mode. Array of tool names, for example `terminalLastCommand` or `githubRepo`. The tool name is shown when you type `#` in the chat input field. If a given tool is not available, it is ignored when running the prompt.
`description`	A short description of the prompt.

- Body with the prompt content
  Prompt files mimic the format of writing prompts in chat. This allows blending natural language instructions, additional context, and even linking to other prompt files as dependencies. You can use Markdown formatting to structure the prompt content, including headings, lists, and code blocks.

You can reference other prompt files or instruction files by using Markdown links. Use relative paths to reference these files, and ensure that the paths are correct based on the location of the prompt file.

Within a prompt file, you can reference variables by using the `${variableName}` syntax. You can reference the following variables:

- Workspace variables - `${workspaceFolder}`, `${workspaceFolderBasename}`
- Selection variables - `${selection}`, `${selectedText}`
- File context variables - `${file}`, `${fileBasename}`, `${fileDirname}`, `${fileBasenameNoExtension}`
- Input variables - `${input:variableName}`, `${input:variableName:placeholder}` (pass values to the prompt from the chat input field)

Prompt file examples

- Prompt for a reusable task to generate a React form:

```
---
mode: 'agent'
tools: ['githubRepo', 'codebase']
description: 'Generate a new React form component'
---
Your goal is to generate a new React form component based on the templates in #githubR
epo contoso/react-templates.

Ask for the form name and fields if not provided.

Requirements for the form:
* Use form design system components: [design-system/Form.md](../docs/design-system/For
m.md)
* Use `react-hook-form` for form state management:
* Always define TypeScript types for your form data
* Prefer *uncontrolled* components using register
* Use `defaultValues` to prevent unnecessary rerenders
* Use `yup` for validation:
* Create reusable validation schemas in separate files
* Use TypeScript types to ensure type safety
* Customize UX-friendly validation rules
```

Copy

- Prompt for performing a security review of a REST API:

```
---
mode: 'edit'
description: 'Perform a REST API security review'
---
Perform a REST API security review:

* Ensure all endpoints are protected by authentication and authorization
* Validate all user inputs and sanitize data
* Implement rate limiting and throttling
* Implement logging and monitoring for security events
```

Copy

> **Tip**
> Reference additional context files like API specs or documentation by using Markdown links to provide Copilot with more complete information.

Create a workspace prompt file

Workspace prompt files are stored in your workspace and are only available in that workspace.

By default, prompt files are located in the `.github/prompts` directory of your workspace. You can specify additional prompt file locations with the `chat.promptFilesLocations (vscode://settings/chat.promptFilesLocations)` setting.

To create a workspace prompt file:

1. Run the **Chat: New Prompt File** command from the Command Palette (⇧⌘P).

2. Choose the location where the prompt file should be created.

   By default, only the `.github/prompts` folder is available. Add more prompt folders for your workspace with the ⚙ `chat.promptFilesLocations (vscode://settings/chat.promptFilesLocations)` setting.

3. Enter a name for your prompt file.

   Alternatively, you can directly create a `.prompt.md` file in the prompts folder of your workspace.

4. Author the chat prompt by using Markdown formatting.

   Within a prompt file, reference additional workspace files as Markdown links (`[index](../index.ts)`), or as `#index.ts` references within the prompt file.

   You can also reference other `.prompt.md` files to create a hierarchy of prompts. You can also reference instructions files in the same way.

Create a user prompt file

User prompt files are stored in your user profile (/docs/configure/profiles). With user prompt files, you can share reusable prompts across multiple workspaces.

To create a user prompt file:

1. Select the **Chat: New Prompt File** command from the Command Palette (⇧⌘P).

2. Select **User Data Folder** as the location for the prompt file.

   If you use multiple VS Code profiles (/docs/configure/profiles), the prompt file is created in the current profile's user data folder.

3. Enter a name for your prompt file.

4. Author the chat prompt by using Markdown formatting.

   You can also reference other user prompt files or user instruction files.

Sync user prompt files across devices

VS Code can sync your user prompt files across multiple devices by using Settings Sync (/docs/configure/settings-sync).

To sync your user prompt files, enable Settings Sync for prompt and instruction files:

1. Make sure you have Settings Sync (/docs/configure/settings-sync) enabled.

2. Run **Settings Sync: Configure** from the Command Palette (⇧⌘P).

3. Select **Prompts and Instructions** from the list of settings to sync.

Use a prompt file in chat

You have multiple options to run a prompt file:

- Run the **Chat: Run Prompt** command from the Command Palette (⇧⌘P) and select a prompt file from the Quick Pick.
- In the Chat view, type `/` followed by the prompt file name in the chat input field.
  This option enables you to pass additional information in the chat input field. For example, `/create-react-form` or `/create-react-form: formName=MyForm` .
- Open the prompt file in the editor, and press the play button in the editor title area. You can choose to run the prompt in the current chat session or open a new chat session.
  This option is useful for quickly testing and iterating on your prompt files.

Settings

Custom instructions settings

- ⚙ `chat.promptFiles` (`vscode://settings/chat.promptFiles`) *(Experimental)*: enable reusable prompt and instruction files.
- ⚙ `github.copilot.chat.codeGeneration.useInstructionFiles` (`vscode://settings/github.copilot.chat.codeGeneration.useInstructionFiles`): controls whether code instructions from `.github/copilot-instructions.md` are added to Copilot requests.
- ⚙ `chat.instructionsFilesLocations` (`vscode://settings/chat.instructionsFilesLocations`) *(Experimental)*: a list of folders where instruction files are located. Relative paths are resolved from the root folder(s) of your workspace. Supports glob patterns for file paths.

Setting value	Description
`["/path/to/folder"]`	Enable instruction files for a specific path. Specify one or more folders where instruction files are located. Relative paths are resolved from the root folder(s) of your workspace. By default, `.github/copilot-instructions` is added but disabled.

- ⚙ `github.copilot.chat.codeGeneration.instructions` (`vscode://settings/github.copilot.chat.codeGeneration.instructions`) *(Experimental)*: set of instructions that will be added to Copilot requests that generate code.
- ⚙ `github.copilot.chat.testGeneration.instructions` (`vscode://settings/github.copilot.chat.testGeneration.instructions`) *(Experimental)*: set of instructions that will be added to Copilot requests that generate tests.
- ⚙ `github.copilot.chat.reviewSelection.instructions` (`vscode://settings/github.copilot.chat.reviewSelection.instructions`) *(Preview)*: set of instructions that will be added to Copilot requests for reviewing the current editor selection.
- ⚙ `github.copilot.chat.commitMessageGeneration.instructions` (`vscode://settings/github.copilot.chat.commitMessageGeneration.instructions`) *(Experimental)*: set of instructions that will be added to Copilot requests that generate commit messages.

⚙ `github.copilot.chat.pullRequestDescriptionGeneration.instructions` (`vscode://settings/github.copilot.chat.pullRequestDescriptionGeneration.instructions`) *(Experimental)*: set of instructions that will be added to Copilot requests that generate pull request titles and descriptions.

Prompt files (experimental) settings

- ⚙ `chat.promptFiles` (`vscode://settings/chat.promptFiles`) *(Experimental)*: enable reusable prompt and instruction files.
- ⚙ `chat.promptFilesLocations` (`vscode://settings/chat.promptFilesLocations`) *(Experimental)*: a list of folders where prompt files are located. Relative paths are resolved from the root folder(s) of your workspace. Supports glob patterns for file paths.

Setting value	Description
`["/path/to/folder"]`	Enable prompt files for a specific path. Specify one or more folders where prompt files are located. Relative paths are resolved from the root folder(s) of your workspace. By default, `.github/prompts` is added but disabled.

Related content

- Get started with chat in VS Code (/docs/copilot/chat/copilot-chat)

05/08/2025

(https://www.microsoft.com)

(https://go.microsoft.com/fwlink/?LinkID=533687)

(https://github.com/microsoft/vscode)

(https://www.youtube.com/@code)

Support (https://support.serviceshub.microsoft.com/supportforbusiness/create?sapId=d66407ed-3967-b000-4cfb-2c318cad363d)

Privacy (https://go.microsoft.com/fwlink/?LinkId=521839)

Terms of Use (https://www.microsoft.com/legal/terms-of-use) License (/License)